



Sandya Mannarswamy

This month's column contains the solutions to some of the programming-related interview questions we featured in the last issue.

Many readers have written back with answers to the questions we featured in last month's column. Others have mentioned that they found some of the questions challenging. So we will keep some of the difficult questions open till next month, giving readers more time to come up with the solutions themselves. However, in this month's column, let us discuss some of the easier ones. Please do send your answers to the other questions so that I can feature your solutions in next month's column.

(1) Can you have both const and volatile qualifiers applied to a single declaration in C? If so, can you give an example? If not, why not?

The surprising answer is that both const and volatile can be applied to a single declaration. Const and volatile are type qualifiers. Const means that the program cannot modify the value of a given variable qualified by a const qualifier. For instance, `const int NUM = 100` is a valid C declaration, which means that NUM is an integer variable whose value cannot be modified by the program. The qualifier volatile means that the value of the variable can change by means that are external to the program, such as by means of an I/O device.

Consider a variable declared as `const volatile int hw_register`; The const declaration means that the value of `hw_register` can not be changed by this program, whereas the 'volatile' specifier implies that its value can be changed at any time from outside the program. However, remember that 'volatile' prevents the compiler from many of its optimisations and hence an injudicious use of 'volatile' can have an impact on your application's performance.

(2) Consider the following code snippet:

```
main()
{
    int a, b;
    a = foo();
    b = foo();
    printf("a = %d b = %d\n", a, b);
}
```

You do not know anything about function 'foo' except that it returns an integer. Are the values of `a` and `b` that are printed by the `printf` function, the same? If not, can you give an example of a function `foo` that will cause `a` and `b` to be different?

No, you can't make any assumption about the values

returned by `foo` by the two invocations. For instance, consider the following definition of `foo()`:

```
int foo(void)
{
    static int counter = 0;
    return counter++;
}
```

In this case, the first invocation of `foo` will return 0 and the second invocation will return 1. Since you do not know how the internal state of `foo` will be affected by each of its invocations, we cannot make any assumptions about the values returned by the two successive invocations of `foo` with the same value of arguments, though they come immediately, one after the other, with no intervening statements.

(3) Given a word, can you find all the anagrams of that word? If you were asked to solve this problem for only one word, what would your solution be? If you were asked to find the anagrams for 10,000 words, would your solution change?

This is a problem we have discussed in one of the earlier columns. The brute force solution of finding the number of anagrams consists of forming each permutation of the word and then looking up that particular permutation in a dictionary to see if it is a valid word. While this may be possible for a single word, repeating this for thousands of words will take too long to be practical. Hence the best thing would be to first build a set of anagram classes of your dictionary so that given a word, you can look up the dictionary for all its anagram classes in a single shot.

This problem is discussed in detail in Chapter 2 of 'Programming Pearls' by John Bentley. The basic idea is to create a signature for each word by sorting it in alphabetical order. For example, both the words 'pans' and 'span' have the signature 'ansp'. Then sort all the words by their signatures to form the anagram classes.

(4) What is the time complexity of searching for an element in each of the following:

- (a) A linked list containing N elements
- (b) A hash table containing N elements
- (c) A binary search tree containing N elements
- (d) A binary heap containing N elements
- (e) A d-heap containing N elements

In a linked list, if you are looking for an element that is not